UNIT 4 · TOPIC 4.1

Device Vulnerabilities and Attacks

Assignment, Answer Key

Teacher answer key, 30 points total.

Practice Questions

4.1.a · Malware types (viruses, worms, trojans, ransomware)

1. A malware sample spreads from device to device over the network with no user interaction and no host file. Which category best fits? (1 pt)

- ○ A. Virus
- **● B. Worm**
- ○ C. Trojan
- ○ D. Ransomware

Why: Self-propagation over a network without user action or a host file is the defining trait of a worm.

2. A user installs a free video converter that secretly opens a remote-control backdoor. Which malware type is this? (1 pt)

- ○ A. Virus
- ○ B. Worm
- ○ C. Ransomware
- **● D. Trojan**

Why: Disguising malicious functionality inside apparently legitimate software is the defining trait of a trojan.

3. Select ALL statements that are TRUE about ransomware. (2 pts)

- **■ A. It encrypts victim files and demands payment for the decryption key**
- □ B. It self-replicates across the network without user action
- **■ C. Offline backups directly neutralize its leverage**
- □ D. It always permanently destroys the encrypted files

Why: Ransomware encrypts (not destroys) files and demands payment; offline backups let victims recover without paying. Ransomware does not self-propagate like a worm.

4. Which property MOST distinguishes a virus from a worm? (1 pt)

- ○ A. Viruses encrypt files; worms do not
- **● B. Viruses require a host file and user action to spread; worms spread autonomously**
- ○ C. Viruses target networks; worms target single devices
- ○ D. Viruses are newer than worms

Why: The core distinction is propagation mechanism: viruses need a host file and user action; worms are self-contained and spread without either.

5. Name the term for the initial path by which malware enters a device (e.g., a phishing email or infected USB drive). (1 pt)

Acceptable answers: infection vector; vector; attack vector; entry vector

Why: The infection vector is the initial entry path, common examples include phishing emails, drive-by downloads, USB drives, and unpatched software exploits.

6. An analyst sees that a device's CPU is near 100% and it is sending data to hundreds of IP addresses, yet no user application is open. Which malware type does this MOST suggest? (1 pt)

○ A. Ransomware

● **B. Worm**

○ C. Virus

○ D. Trojan

Why: Autonomous network scanning and self-copying, consuming CPU and bandwidth without user action, is characteristic worm behavior.

4.1.b · Device tampering and physical compromise

7. A security analyst discovers a small adapter connected between a desktop computer's USB port and the keyboard cable. No software alerts have been triggered. What type of physical attack is this? (1 pt)

○ A. USB Rubber Ducky

○ B. Cold boot attack

● **C. Hardware keylogger**

○ D. BIOS malware

Why: A hardware keylogger sits between the keyboard and USB port to log typing physically without software detection.

8. Which of the following mitigations specifically address threats arising from physical theft of a laptop hard drive? (2 pts)

■ **A. Full Disk Encryption (FDE)**

□ B. Strong operating system logon passwords

■ **C. Trusted Platform Module (TPM) integration**

□ D. Software-based firewalls

Why: Full Disk Encryption (FDE) prevents an attacker from reading data off a stolen drive, and TPM checks boot integrity before releasing keys. OS login passwords and local firewalls can be bypassed if the hard drive is physically removed and read on a different system.

9. Name the keystroke injection tool disguised as a standard USB flash drive that executes pre-programmed commands by mimicking a keyboard. (1 pt)

Acceptable answers: usb rubber ducky; rubber ducky; keystroke injector; keystroke injection tool

Why: A USB Rubber Ducky is a classic physical attack tool that mimics a keyboard to run commands at high speed.

10. An attacker sprays compressed air (upside down) on RAM chips immediately after a workstation is powered off, then quickly moves the modules to another machine to read their contents. What is this attack called? (1 pt)

○ A. Chassis intrusion

○ B. Hardware keylogger

○ C. USB Rubber Ducky

● **D. Cold boot attack**

Why: A cold boot attack exploits the fact that RAM contents fade gradually rather than instantly; cooling the chips slows that decay long enough to extract recently-loaded keys or passwords.

11. A server room has a keypad lock on its door, but an attacker who is already an employee with legitimate badge access removes a hard drive and reads it on a personal laptop. Which control would have stopped this? (1 pt)

- ○ A. A stronger door lock
- ○ B. A longer OS login password
- ○ C. A security camera in the lobby
- **● D. Full Disk Encryption (FDE)**

Why: Room-level physical locks only stop people who lack access to the room. FDE protects the data itself so that even someone who can physically remove the drive cannot read it without the decryption key.

12. Name the term for a sticker or label on a device's case or port that visibly tears or changes color if removed, letting staff confirm at a glance that the device has not been opened. (1 pt)

Acceptable answers: tamper-evident seal; tamper evident seal; tamper seal; tamper-proof seal

Why: Tamper-evident seals provide a low-cost, visual way to detect physical tampering after the fact, commonly used on ATMs and point-of-sale terminals.

4.1.c · IoT device vulnerabilities

13. A smart camera is still logging in with the username/password it shipped with from the factory. What is this vulnerability called? (1 pt)

- ○ A. Outdated firmware
- ○ B. Unauthenticated API
- ○ C. Shadow IoT
- **● D. Default credentials**

Why: Default credentials are factory-set login details that were never changed by the owner; attackers scan for and exploit them automatically.

14. Why is the 2016 Mirai botnet a significant example in IoT security? (1 pt)

- ○ A. It was the first computer virus ever discovered
- ○ B. It encrypted victim files and demanded a ransom
- ○ C. It only affected desktop computers, not IoT devices
- **● D. It hijacked IoT devices via default Telnet credentials and used them to launch record-setting DDoS attacks**

Why: Mirai scanned the internet for IoT devices still using default Telnet credentials, enlisted them into a botnet, and used the combined force for massive DDoS attacks.

15. Select ALL statements that are TRUE about IoT device risk. (2 pts)

- **■ A. A device that isn't directly internet-facing can still be reached by other devices already on the same local network**
- □ B. Manufacturers always issue timely firmware updates for the life of the product
- **■ C. UPnP can automatically open a port on a router, exposing a device without the owner realizing it**
- **■ D. Attackers often want IoT devices as botnet nodes or network pivot points, not for the device's own data**

Why: Devices on a shared network can attack each other regardless of internet exposure; UPnP can silently expose management interfaces; attackers value IoT devices mainly for scale (botnets) and as pivot points. Many manufacturers do NOT maintain updates for the full product lifetime, devices often become end-of-life (EOL).

16. Which property MOST distinguishes an EOL (end-of-life) IoT device's risk from a device with an exposed management port? (1 pt)

- ○ A. EOL devices no longer connect to Wi-Fi at all
- ○ B. EOL devices always use encrypted traffic by default
- **● C. EOL devices no longer receive firmware updates, so any known vulnerability stays permanently unpatched, regardless of ports or credentials**
- ○ D. EOL devices cannot be part of a botnet

Why: End-of-life status means the manufacturer has stopped issuing updates, so any vulnerability discovered in the firmware is never fixed for the rest of the device's life.

17. Name the term for devices connected to a network without IT's knowledge, approval, or security oversight. (1 pt)

Acceptable answers: shadow iot; shadow it; shadow devices

Why: Shadow IoT refers to unapproved, unmonitored devices on a network that silently expand the attack surface.

18. A device's mobile app sends commands to it over the local Wi-Fi network with no encryption at all. What is the MOST direct consequence? (1 pt)

- ○ A. The device automatically becomes part of a botnet
- ○ B. The device's firmware will stop receiving updates
- **● C. Anyone else on the same network segment can read or alter the traffic in transit**
- ○ D. The manufacturer's cloud API becomes unauthenticated

Why: Unencrypted traffic can be intercepted (read) or tampered with (altered) by anyone who can observe the same network segment.

Coding Assignments (Python 3)

4.1.a · Malware types (viruses, worms, trojans, ransomware)

3 pts · auto-graded on the site

Coding problem, 4.1.a: Classify malware by behavioral signature

Skill: Analyze Risk · **Maps to:** 4.1.A, 4.1.B · **Language:** Python 3 (pytest)

Problem

A security analyst receives short incident-report strings that describe observed device behavior. Build the classifier that maps each description to the correct malware category.

Categories:

- `"virus"`, attaches to host files; spreads when files are shared or opened
- `"worm"`, self-propagates across the network without user action; autonomous scanning
- `"trojan"`, disguised as legitimate software; creates backdoor / exfiltrates data
- `"ransomware"`, encrypts files; demands payment for decryption key

A report may match exactly one category. If no category matches, return `"unknown"`.

Function contract

Implement `solve(report: str) -> str` in `starter.py` (see `interface.md` / `blackbox.yaml`). Return the single category label string. Matching is case-insensitive substring detection using the indicator sets defined in `interface.md`.

Input

A single string describing observed behavior on a device or network.

Output

One of: `"virus"`, `"worm"`, `"trojan"`, `"ransomware"`, `"unknown"`.

Examples

```
"Device is scanning all ports on the subnet with no user logged in"         -> "worm"
"All .docx files have been encrypted; a ransom note appeared on the desktop" -> "ransomware"
"A free game installer is sending keystrokes to a remote server"            -> "trojan"
"An email attachment infected the file and spread copies to shared folders" -> "virus"
"Battery usage is higher than usual after a long meeting"                   -> "unknown"
```

Constraints

- Deterministic, offline, standard library only.
- Detection by case-insensitive substring match against indicator sets in `interface.md`.
- Exactly one label returned (string, not list).
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Reference solution:

```
"""Reference solution for 4.1.a. MUST pass 100% of tests.py.
Also the default 'blackbox' implementation students can swap out (see blackbox.yaml)."""


def solve(report: str) -> str:
    """Classify malware by behavioral signature report.

    Args:
        report: A string containing the incident report description.

    Returns:
        The malware category string: "virus", "worm", "trojan", "ransomware", or "unknown".
    """
    if not isinstance(report, str):
        return "unknown"

    report_lower = report.lower()

    # Priority order: worm > ransomware > trojan > virus > unknown

    # 1. worm
    worm_indicators = [
        "scanning", "self-propagat", "autonomous", "no user action",
        "spreading across network", "network scan", "subnet scan",
        "port scan", "exploiting vulnerab"
```

```
    ]
    if any(ind in report_lower for ind in worm_indicators):
        return "worm"

    # 2. ransomware
    ransomware_indicators = [
        "encrypt", "ransom", "decryption key", "locked", ".locked",
        "bitcoin", "btc", "pay to recover", "files inaccessible"
    ]
    if any(ind in report_lower for ind in ransomware_indicators):
        return "ransomware"

    # 3. trojan
    trojan_indicators = [
        "backdoor", "keylog", "exfiltrat", "disguised", "fake installer",
        "remote control", "command and control", "c2", "sending keystrokes",
        "free crack", "pirated"
    ]
    if any(ind in report_lower for ind in trojan_indicators):
        return "trojan"

    # 4. virus
    virus_indicators = [
        "host file", "infected file", "macro", "attachment infected",
        "spread copies", "shared folder infect", "email attachment",
        "copied to usb", "file infect"
    ]
    if any(ind in report_lower for ind in virus_indicators):
        return "virus"

    return "unknown"
```

4.1.b · Device tampering and physical compromise

3 pts · auto-graded on the site

Coding problem, 4.1.b: Device tampering and physical compromise

Skill: Analyze Risk · **Maps to:** 4.1.B · **Language:** Python 3 (pytest)

Problem

In a physical security audit, we collect logs from system motherboards, BIOS events, and OS events. You must implement a parser that reads these logs and identifies all compromised devices.

A device is considered physically compromised if it triggers any of the following rules: 1. **Chassis Intrusion:** `event_type == "chassis_intrusion"` and `details` contains `"open"` (case-insensitive). 2. **Unauthorized Boot:** `event_type == "boot"` and `details` contains `"usb"`, `"external"`, or `"live"` (case-insensitive). 3. **USB Rubber Ducky:** A `usb_insert` event with `"keyboard"` or `"hid"` in `details` (case-insensitive) followed by a `command` event on the same device within 3 seconds, where the command `details` contains `"powershell"`, `"cmd.exe"`, or `"download"` (case-insensitive).

Function contract

Implement `solve(logs: list) -> list` in `starter.py` (see `interface.md` / `blackbox.yaml`). Return the sorted list of compromised device IDs.

Input

A list of dicts:

```
[
    {
        "timestamp": "2026-06-25T12:00:00Z",
```

```
        "device_id": "laptop_01",
        "event_type": "boot",
        "details": "Booted from external USB drive"
    }
]
```

Output

A list of strings, e.g. `["laptop_01"]`, sorted alphabetically.

Examples

```
solve([
    {"timestamp": "2026-06-25T10:00:00Z", "device_id": "pc_01", "event_type": "chassis_intrusion", "details":
"chassis opened detected"}
]) -> ["pc_01"]

solve([
    {"timestamp": "2026-06-25T10:00:00Z", "device_id": "pc_02", "event_type": "usb_insert", "details": "USB
Keyboard connected"},
    {"timestamp": "2026-06-25T10:00:02Z", "device_id": "pc_02", "event_type": "command", "details": "powershell
-nop -w hidden -c Get-Process"}
]) -> ["pc_02"]
```

Constraints

- Pure function, offline, standard library only.
- Timestamps are ISO-8601 strings.
- Sorted return list with unique elements.

Reference solution:

```
"""Reference solution for 4.1.b. MUST pass 100% of tests.py.
Also the default 'blackbox' implementation students can swap out (see blackbox.yaml)."""
from datetime import datetime


def solve(logs: list) -> list:
    """Analyze logs to detect physical tampering and device compromise.

    Args:
        logs: A list of log dictionaries, sorted chronologically.

    Returns:
        A list of compromised device IDs, sorted alphabetically.
    """
    compromised = set()
    if not logs:
        return []

    def parse_time(ts_str):
        return datetime.fromisoformat(ts_str.replace("Z", "+00:00"))

    # For fast lookup of USB insertions per device
    usb_inserts = []  # List of (timestamp_dt, device_id)

    # Process all events
    for event in logs:
        device_id = event.get("device_id")
        event_type = event.get("event_type", "")
        details = event.get("details", "")
```

```
        ts_str = event.get("timestamp", "")

        if not device_id or not ts_str:
            continue

        try:
            ts_dt = parse_time(ts_str)
        except ValueError:
            continue

        details_lower = details.lower()

        # Rule 1: Chassis Intrusion
        if event_type == "chassis_intrusion" and "open" in details_lower:
            compromised.add(device_id)

        # Rule 2: Unauthorized Boot
        elif event_type == "boot" and any(k in details_lower for k in ["usb", "external", "live"]):
            compromised.add(device_id)

        # Record USB insert for Rule 3
        elif event_type == "usb_insert" and any(k in details_lower for k in ["keyboard", "hid"]):
            usb_inserts.append((ts_dt, device_id))

        # Check Command for Rule 3
        elif event_type == "command":
            for usb_ts, usb_dev in usb_inserts:
                if usb_dev == device_id:
                    time_diff = (ts_dt - usb_ts).total_seconds()
                    if 0 <= time_diff <= 3:
                        if any(k in details_lower for k in ["powershell", "cmd.exe", "download"]):
                            compromised.add(device_id)

    return sorted(list(compromised))
```

4.1.c · IoT device vulnerabilities

3 pts · auto-graded on the site

Coding problem, 4.1.c: Audit an IoT device for its dominant vulnerability

Skill: Analyze Risk · **Maps to:** 4.1.A, 4.1.C · **Language:** Python 3 (pytest)

Problem

A security analyst is auditing smart devices on a network. Each device is described by a dictionary of attributes. Build the classifier that inspects a device and returns its single most severe vulnerability category, or `"secure"` if none of the checked issues are present.

A device can technically have more than one issue at once; when it does, report only the most severe one, using this fixed priority order (highest severity first):

1. `"default_credentials"`, the device still uses its factory default password 2. `"outdated_firmware"`, the device's firmware is out of date 3. `"exposed_port"`, the device has an insecure management port open (23 = Telnet, 21 = FTP) 4. `"unauthenticated_api"`, the device's API accepts requests with no authentication 5. `"unencrypted_traffic"`, the device's network traffic is not encrypted 6. `"secure"`, none of the above apply

Function contract

Implement `solve(device: dict) -> str` in `starter.py` (see `interface.md` / `blackbox.yaml`). Return the single verdict label string.

Input

A dictionary with the following keys:

- `default_password` (bool), True if the factory default password is still in use
- `firmware_outdated` (bool), True if the firmware is out of date
- `open_ports` (list[int]), list of open network ports on the device
- `unauthenticated_api` (bool), True if the device's API requires no authentication
- `encrypted_traffic` (bool), True if the device's traffic is encrypted (note: False means traffic is UNencrypted, which is the risk condition)

An insecure management port is any of 23 (Telnet) or 21 (FTP) appearing in `open_ports`.

Output

One of: `"default_credentials"`, `"outdated_firmware"`, `"exposed_port"`, `"unauthenticated_api"`, `"unencrypted_traffic"`, `"secure"`.

Examples

```
{"default_password": True, "firmware_outdated": True, "open_ports": [23], "unauthenticated_api": True,
"encrypted_traffic": False}  ->  "default_credentials"
{"default_password": False, "firmware_outdated": True, "open_ports": [443], "unauthenticated_api": False,
"encrypted_traffic": True}  ->  "outdated_firmware"
{"default_password": False, "firmware_outdated": False, "open_ports": [23, 443], "unauthenticated_api": False,
"encrypted_traffic": True}  ->  "exposed_port"
{"default_password": False, "firmware_outdated": False, "open_ports": [443], "unauthenticated_api": True,
"encrypted_traffic": True}  ->  "unauthenticated_api"
{"default_password": False, "firmware_outdated": False, "open_ports": [443], "unauthenticated_api": False,
"encrypted_traffic": False}  ->  "unencrypted_traffic"
{"default_password": False, "firmware_outdated": False, "open_ports": [443], "unauthenticated_api": False,
"encrypted_traffic": True}  ->  "secure"
```

Constraints

- Deterministic, offline, standard library only.
- Priority order is fixed as listed above and must be applied exactly.
- Only ports 23 and 21 count as insecure/exposed management ports.
- Exactly one label returned (string, not list).
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Reference solution:

```
"""Reference solution for 4.1.c. MUST pass 100% of tests.py.
Also the default 'blackbox' implementation students can swap out (see blackbox.yaml)."""

INSECURE_PORTS = {23, 21}
```

```
def solve(device: dict) -> str:
    """Audit an IoT device dict and return its single dominant vulnerability verdict.

    Args:
        device: dict with keys default_password (bool), firmware_outdated (bool),
            open_ports (list[int]), unauthenticated_api (bool), encrypted_traffic (bool).
            Missing keys default to the "safe" value for that key.

    Returns:
        One of: "default_credentials", "outdated_firmware", "exposed_port",
        "unauthenticated_api", "unencrypted_traffic", "secure".
    """
    if not isinstance(device, dict):
        return "secure"

    default_password = bool(device.get("default_password", False))
    firmware_outdated = bool(device.get("firmware_outdated", False))
    open_ports = device.get("open_ports", []) or []
    unauthenticated_api = bool(device.get("unauthenticated_api", False))
    encrypted_traffic = bool(device.get("encrypted_traffic", True))

    # Priority order: default_credentials > outdated_firmware > exposed_port >
    # unauthenticated_api > unencrypted_traffic > secure

    if default_password:
        return "default_credentials"

    if firmware_outdated:
        return "outdated_firmware"

    if any(port in INSECURE_PORTS for port in open_ports):
        return "exposed_port"

    if unauthenticated_api:
        return "unauthenticated_api"

    if not encrypted_traffic:
        return "unencrypted_traffic"

    return "secure"
```